PRODUCT REQUIREMENTS DOCUMENT: MNIST Digit Recognizer

Version: 1.1 (Updated)

Date: May 11, 2026

Status: Active Development

Project Type: Deep Learning / Multi-class Image Classification

---

1. Project Overview

This project builds a **Convolutional Neural Network (CNN)** to classify handwritten digits (0–9) from 28×28 grayscale images. It covers the full machine learning lifecycle, from raw pixel processing to serving a model via a **FastAPI REST API** and an interactive **HTML5 canvas** drawing app.

1.1 Objectives

- **Performance:** Achieve $\ge 99\%$ test accuracy.
- **Architecture Comparison:** Implement DigitCNN (2-layer) and DigitCNNBN (with Batch Normalization).
- **Evaluation:** Generate a 10×10 confusion matrix and save a detailed classification_report.txt.
- **Deployment:** serve the model at a /predict endpoint and provide a real-time browser interface.
- **Testing:** Maintain code quality through a comprehensive pytest suite.

---

2. Dataset & Preprocessing

The **MNIST database** consists of 70,000 grayscale images (28×28 pixels).

2.1 Preprocessing Pipeline

1. **Normalization:** Scale pixels from $[0, 255]$ to $[0.0, 1.0]$ or use mean 0.1307 and std 0.3081.
2. **Reshaping:** Convert to shape $(N, 1, 28, 28)$ for PyTorch (channels-first).
3. **Augmentation (Training Only):** Apply random rotation ($\pm 10^{\circ}$), translation ($\pm 10\%$), and zoom (0.9–1.1).
    - *Note:* Do **not** use horizontal flips, as they invalidate digits like 6 and 9.

---

3. CNN Architecture

The model uses a modular 2-block design. Each block extracts increasingly complex features: edges, then curves, then digit structures.

3.1 Layer Specification

Layer	Type	Config	Output Shape
Input	Grayscale Image	28×28	$1 \times 28 \times 28$
Conv1	Conv2d	32 filters, 3×3, ReLU	$32 \times 26 \times 26$
Pool1	MaxPool2d	2×2, stride 2	$32 \times 13 \times 13$
Conv2	Conv2d	64 filters, 3×3, ReLU	$64 \times 11 \times 11$
Pool2	MaxPool2d	2×2, stride 2	$64 \times 5 \times 5$
Flatten	Flatten	—	1600
Dropout	Dropout	$p = 0.5$	1600
FC1	Linear	128 units, ReLU	128
Output	Linear	10 units + LogSoftmax	10

Total Parameters: ~224,834.

---

4. Technical Stack & Structure

4.1 Core Stack

- **Frameworks:** Python 3.10+, PyTorch 2.0+.
- **Web/API:** FastAPI, Uvicorn, HTML5 Canvas.
- **Data Science:** Scikit-learn, Matplotlib, Seaborn.
- **Testing:** Pytest.

4.2 Project Directory Tree

Plaintext

```
digit-recognizer/
├── data/              # Auto-downloaded MNIST files
├── notebooks/         # 01_eda.ipynb, 02_experiments.ipynb
├── src/               # Core logic
│   ├── dataset.py     # DataLoader & transforms
│   ├── model.py       # CNN & BatchNorm variants
│   ├── train.py       # Training loop
│   ├── evaluate.py    # Error analysis & report generation
│   └── api.py         # FastAPI endpoint
├── static/            # Frontend (index.html)
├── models/            # Saved mnist_cnn.pt weights
├── tests/             # Unit tests & classification_report.txt
└── requirements.txt   # Dependency list
```

---

5. Non-Functional Requirements

- **Latency:** API response time must be $< 200ms$ per prediction.
- **Reproducibility:** All hyperparameters (learning rate, batch size) must be defined as constants at the top of train.py.
- **Portability:** The trained model file size must remain $< 10MB$ (current: $\sim 0.9MB$).
- **Robustness:** The drawing canvas must apply a Gaussian blur to user strokes to match the "soft" edges of the original MNIST training data.

---

6. Evaluation Criteria

1. **Accuracy:** $\ge 99.0\%$ on the 10,000 test images.
2. **Regularization:** Training curves must show a gap of $< 1\%$ between train and validation accuracy.
3. **Deep Analysis:** Identify and document the top confused pairs (typically $4/9$ and $3/8$).
4. **Code Coverage:** Unit tests must pass for model forward pass shapes and API response schemas.